

CH6: ADVANCED TOPICS

Contents

Mobile Databases	1
Distributed Object Management.....	2
Multi-databases (MDBS)	2
Types of MDBS.....	3
1. Based on Autonomy	3
2. Based on Heterogeneity	3
3. Based on Distribution	4
4. Based on Data Model Integration	4
5. Based on System Architecture	5

Mobile Databases

A Mobile Database is a type of distributed database system where clients or users move physically, and the network connection is not stable. The database must support intermittent connectivity, disconnections, and limited bandwidth, commonly in mobile computing environments (e.g., smartphones, tablets, vehicles).

◆ Architecture:

1. **Mobile Host (MH)** – A portable device that runs applications and stores a subset of data.
2. **Fixed Host (FH)** – Traditional stationary servers.
3. **Base Station (BS)** – Acts as a communication interface between mobile and fixed hosts.

◆ Challenges:

- **Frequent Disconnections** due to mobility
- **Limited Resources** (battery, storage, bandwidth)
- **Synchronization** issues between mobile and central databases
- **Security & Privacy** concerns in wireless communication

◆ Solutions and Features:

- **Caching:** Frequently used data is stored locally to reduce server access.
- **Replication:** Data may be partially or fully replicated on mobile devices.
- **Synchronization Techniques:** To sync data when reconnected (e.g., conflict resolution policies).
- **Lightweight Transactions:** Optimized for mobile resources.

Distributed Object Management

Distributed Object Management (DOM) extends DDBMS concepts by using objects (from object-oriented programming) as the basic units of data instead of traditional records and tables.

DOM systems manage distributed object-oriented databases (DOODB) where data and behavior are encapsulated together. **Key Features:**

- **Encapsulation:** Data and operations (methods) are bundled into objects.
- **Object Identity:** Each object has a unique identity, persistent over time and across sites.
- **Inheritance:** Objects can inherit behavior from other classes.
- **Complex Objects:** Can store lists, sets, arrays — more than just simple data types.
- **Transparency:** Users do not need to know the location of objects.

◆ Challenges in Distributed Setup:

- **Object Migration:** Moving objects between sites without breaking references.
- **Object Replication:** Keeping multiple copies consistent.
- **Remote Method Invocation (RMI):** Calling object methods remotely.
- **Concurrency Control:** Handling simultaneous updates on object attributes.
- **Garbage Collection:** Managing deletion across distributed sites.

◆ Technologies Used:

- **CORBA (Common Object Request Broker Architecture)**
- **Java RMI**
- **Object Data Management Group (ODMG) standards**

Multi-databases (MDBS)

A Multidatabase System (MDBS) is a system that provides a unified interface to access multiple autonomous databases that may be:

- Heterogeneous (different data models, query languages)
- Distributed over multiple locations
- Independently managed

Goal: Allow users to query and update **multiple databases** through a **single global schema/interface**, hiding the complexity of differences.

◆ Types of Heterogeneity:

1. **Data Model Heterogeneity** – Different databases (e.g., relational, NoSQL, object-oriented).
2. **Query Language Heterogeneity** – Different languages (SQL, XQuery, etc.)
3. **Semantic Heterogeneity** – Same term used differently across databases.
4. **Schema Heterogeneity** – Structural differences in schemas.

◆ MDBS Architecture:

1. **Local DBMSs** – Existing independent databases.
2. **Global Schema** – Unified representation of all databases.
3. **Middleware Layer** – Handles query translation, decomposition, and integration.

◆ Functions of MDBS:

- **Query Decomposition:** Breaking a global query into local subqueries.

- **Result Integration:** Merging results from different sources.
- **Schema Mapping:** Translating queries between global and local schemas.
- **Conflict Resolution:** Handling semantic and structural conflicts.

◆ Advantages:

- **Unified Access:** Single interface for many databases.
- **Autonomy Preservation:** Local databases remain independent.
- **Scalability:** Easy to add/remove databases.

◆ Challenges:

- Maintaining **global consistency**
- **Query optimization** across different DBMSs
- Handling **data security and access control** across organizations

Types of MDBS

1. Based on Autonomy

✓ **Tightly Coupled MDBS:** All participating databases are integrated closely and controlled by a central authority.

- Centralized management.
- Less autonomy for individual databases.
- Often built from scratch or with similar systems.
- **Advantages:**
 - Easier consistency and control.
 - Better performance due to tighter integration.
- **Disadvantages:**
 - Difficult to integrate existing autonomous systems.

✓ **Loosely Coupled MDBS:** Participating databases are **independent and autonomous**, integrated using middleware or a federation system.

- Each database manages its own schema, data, and operations.
- Communication via a coordination layer.
- **Advantages:**
 - Supports heterogeneity and scalability.
 - Easier to plug in existing databases.
- **Disadvantages:**
 - Harder to enforce global constraints and consistency.

2. Based on Heterogeneity

✓ **Homogeneous MDBS:** All databases in the system use the same DBMS, data model, and query language. **Example:** Multiple MySQL servers integrated in a single system.

- **Advantages:**
 - Simplified integration and query processing.
 - Easier optimization and transaction management.
- **Disadvantages:**
 - Limited flexibility (can't integrate different types of systems).

✓ **Heterogeneous MDBS:** Databases use different DBMSs, data models (e.g., relational, object-oriented), and query languages (e.g., SQL, XQuery). **Example:** Combining Oracle (relational) and MongoDB (NoSQL).

- **Advantages:**
 - Greater flexibility to integrate diverse systems.
- **Disadvantages:**
 - Complex query translation and schema mapping.
 - More effort to ensure semantic consistency.

3. Based on Distribution

✓ **Federated Databases:** A type of loosely coupled MDBS where multiple databases are **linked together under a common framework** (federation).

- Each database retains autonomy.
- Users can access and query across all databases via a global interface.
- **Types:**
 - **Strong Federation:** Shared data dictionary and control.
 - **Weak Federation:** Minimal coordination, more autonomy.
- **Advantages:**
 - Supports local autonomy while providing global access.
- **Disadvantages:**
 - Complex query planning and transaction management.

✓ **Non-Federated MDBS:** A collection of databases with **no formal federation**, often connected via application-level integration.

- No unified schema or dictionary.
- May lack transparency.
- **Advantages:**
 - Quick integration of existing systems.
- **Disadvantages:**
 - Poor query support and coordination.
 - No standard interface.

4. Based on Data Model Integration

✓ **Global Schema MDBS:** A **single unified schema** is created to represent all the local databases in the system.

- Users write queries based on this global schema.
- The system translates global queries into local subqueries.
- **Advantages:**
 - Simple and intuitive for users.
 - Good data abstraction.
- **Disadvantages:**
 - Difficult to maintain consistency when local schemas change.
 - High complexity in schema design.

✓ **Schema Mapping-based MDBS:** Instead of creating a single global schema, the system uses **mappings** between local schemas to resolve heterogeneity.

- More dynamic and flexible.
- Queries are rewritten using mapping rules.
- **Advantages:**
 - Easier to maintain as systems evolve.
 - Suitable for loosely coupled, heterogeneous systems.
- **Disadvantages:**
 - Complex mapping and query rewriting logic.
 - May affect performance.

5. Based on System Architecture

✓ **Mediator-based MDBS:** Uses a **mediator layer** between the user and data sources to manage communication, integration, and translation.

- Mediator receives the user query, decomposes it, routes it to the right databases, and integrates the results.
- Transparent to users.
- **Advantages:**
 - Simplifies user interaction.
 - Good for heterogeneous environments.
- **Disadvantages:**
 - Performance overhead.
 - Single point of failure if not replicated.

✓ **Agent-based MDBS:** Uses **software agents** (autonomous programs) to represent and interact with individual databases.

- Each agent knows about its own local database.
- Agents cooperate to process user queries.
- **Advantages:**
 - Flexible and adaptive.
 - Suitable for dynamic and mobile environments.
- **Disadvantages:**
 - Coordination among agents can be complex.
 - Security and trust between agents is a concern.